Architecture 1001: x86-64 Assembly BitmaskExample.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

BitmaskExample.c

```c
int main(){
    int a = 0x1301;
    int b = 0x0100;
    if(a & b){
        return 0x70dd1e;
    }
    else{
        return 0x707;
    }
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],1301h
000000014000100C  mov  dword ptr [rsp],100h
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  mov  ecx,dword ptr [rsp+4]
000000014000101A  and  ecx,eax
000000014000101C  mov  eax,ecx
000000014000101E  test eax,eax
0000000140001020  je   000000014000102B
0000000140001022  mov  eax,70DD1Eh
0000000140001027  jmp  0000000140001030
0000000140001029  jmp  0000000140001030
000000014000102B  mov  eax,707h
0000000140001030  add  rsp,18h
0000000140001034  ret
```

BitmaskExample.c

```
int main(){
    int a = 0x1301;
    int b = 0x0100;
    if(a & b){
        return 0x70dd1e;
    }
    else{
        return 0x707;
    }
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],1301h
000000014000100C  mov  dword ptr [rsp],100h
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  mov  ecx,dword ptr [rsp+4]
000000014000101A  and  ecx,eax
000000014000101C  mov  eax,ecx
000000014000101E  test eax,eax
0000000140001020  je   000000014000102B
0000000140001022  mov  eax,70DD1Eh
0000000140001027  jmp  0000000140001030
0000000140001029  jmp  0000000140001030
000000014000102B  mov  eax,707h
0000000140001030  add  rsp,18h
0000000140001034  ret
```

TEST - Logical Compare

- "Computes the bit-wise AND of first operand (source 1 operand) and the second operand (source 2 operand) and sets the SF, ZF, and PF status flags according to the result."
- Like CMP - sets flags, and throws away the result



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

BitmaskExample.c Takeaways

- Conditions depending on bit tests (which is often expressed with boolean logic instructions) will often see the RFLAGS set with the TEST instruction. TEST is like AND, but throws the results away
- The unreachable jmp here is because it's unoptimized code so the compiler is following a simpler set of asm construction rules

```
int main(){
    int a = 0x1301;
    int b = 0x0100;
    if(a & b){
        return 0x70dd1e;
    }
    else{
        return 0x707;
    }
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],1301h
000000014000100C  mov  dword ptr [rsp],100h
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  mov  ecx,dword ptr [rsp+4]
000000014000101A  and  ecx,eax
000000014000101C  mov  eax,ecx
000000014000101E  test eax,eax
0000000140001020  je   000000014000102B
0000000140001022  mov  eax,70DD1Eh
0000000140001027  jmp  0000000140001030
0000000140001029  jmp  0000000140001030
000000014000102B  mov  eax,707h
0000000140001030  add  rsp,18h
0000000140001034  ret
```

Instructions we now know (22)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/**TEST**
- AND/OR/XOR/NOT
- INC/DEC

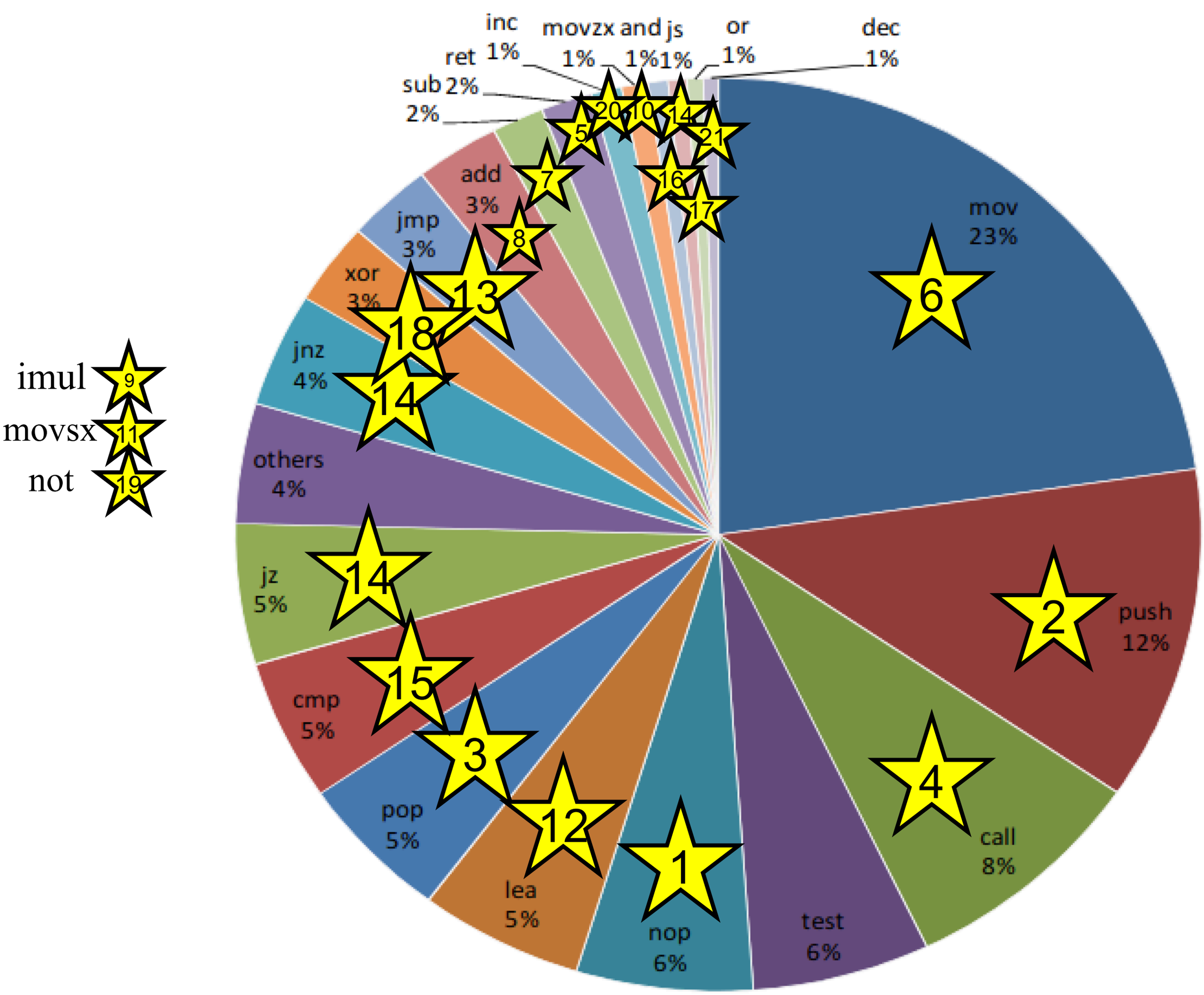


Fig. 2 Web browsers' instruction frequencies

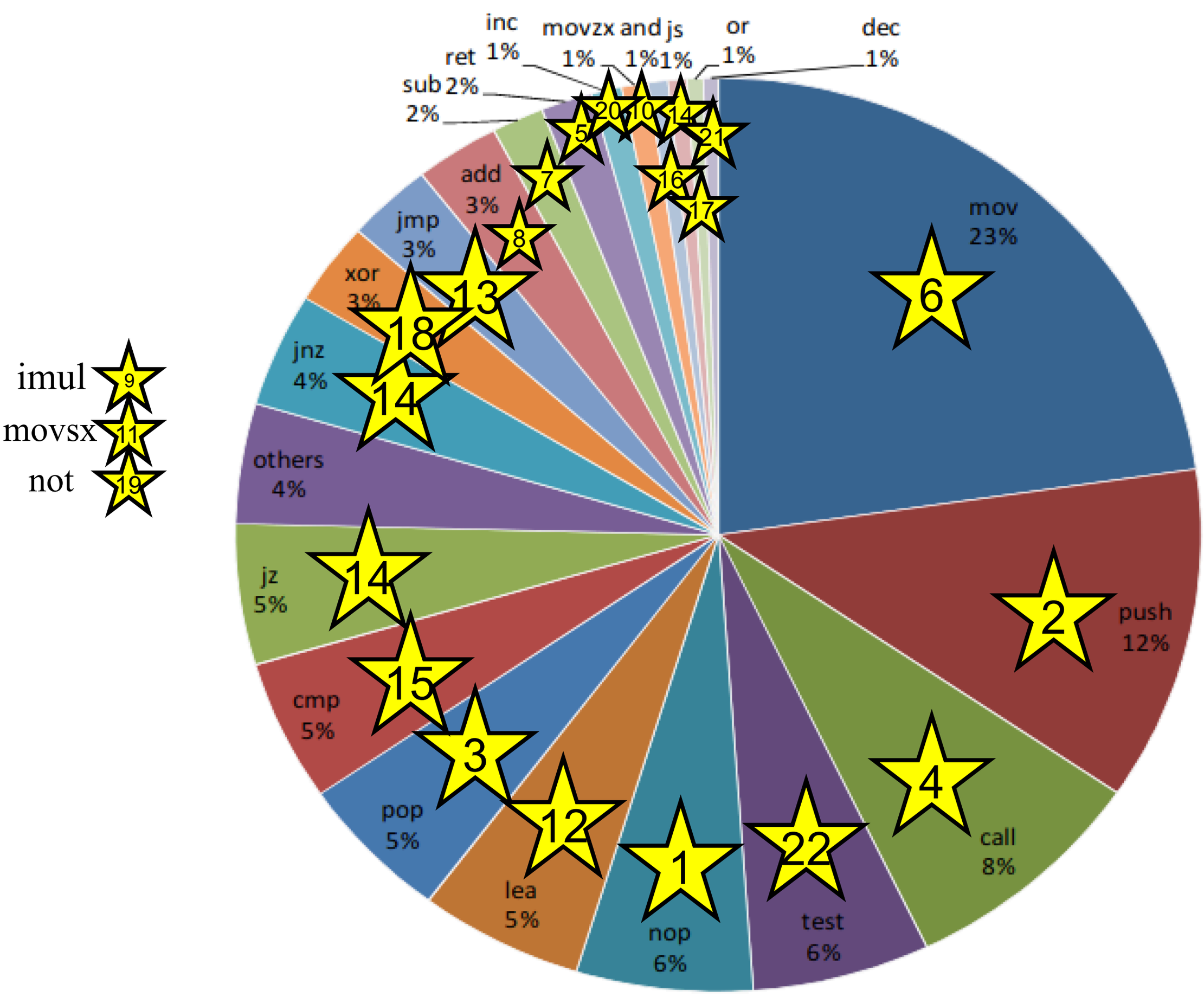


Fig. 2 Web browsers' instruction frequencies

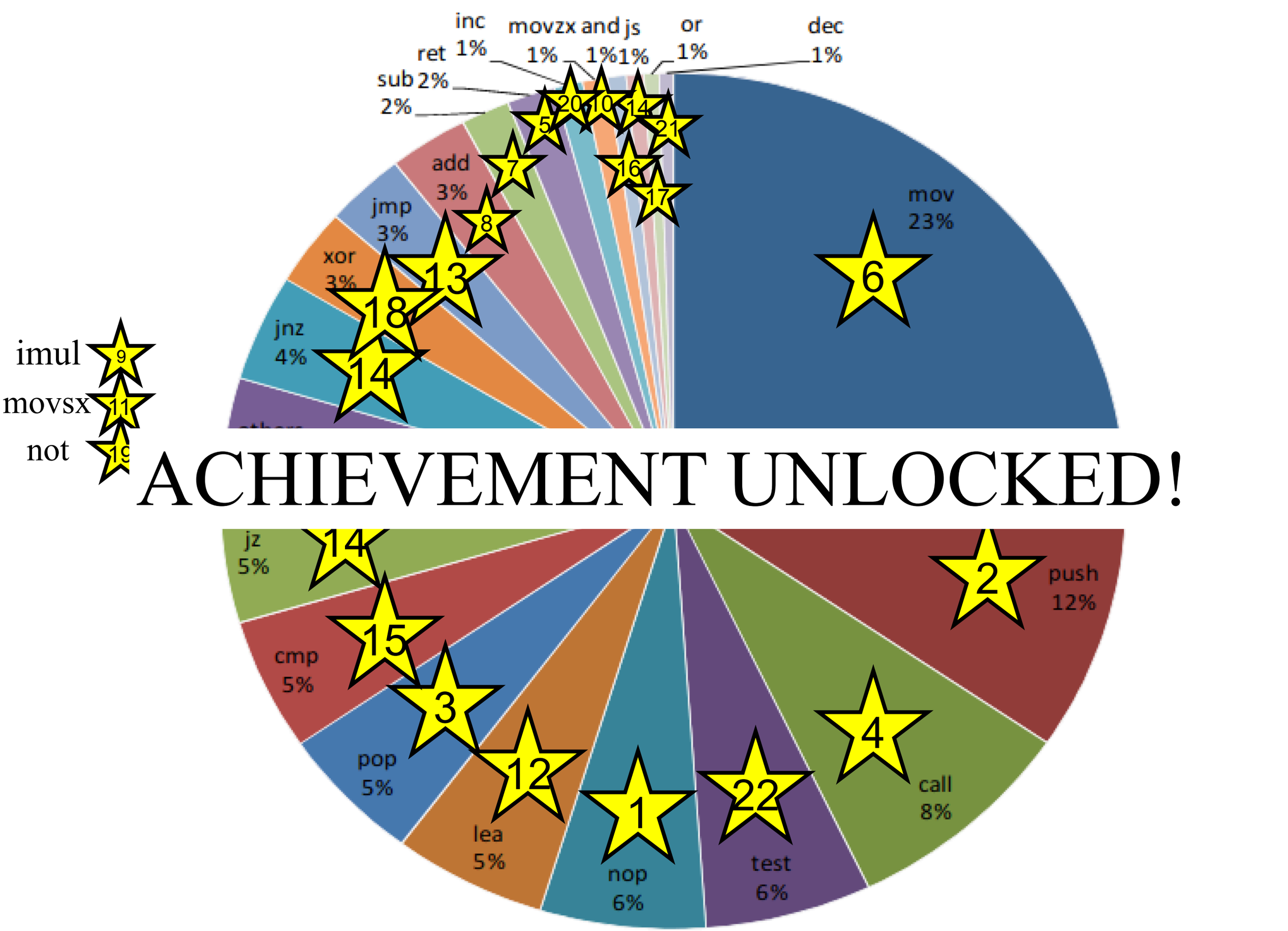


Fig. 2 Web browsers' instruction frequencies

GREAT KID, DON'T GET COCKY

YOU BASICALLY JUST LEARNED HOW TO PRONOUNCE THE ALPHABET!